Exact Design-Space Exploration for SoC Security Feature Synthesis: A Declarative Formulation with Cross-Solver Validation on OpenTitan

Anonymous Submission

***Abstract*—Security features in Systems-on-Chip consume the same LUT, FF, power, and latency budget as the functional design, yet security architecture decisions are still made manually. This paper formulates system-level security feature synthesis as an exact constrained optimization problem using Answer Set Programming (ASP). Given a component and asset model with impact annotations, the method selects a security mechanism and a monitoring option for every component under explicit FPGA resource and per-asset latency constraints, returning provably optimal solutions.**

We validate on a benchmark suite spanning 1 to 20 components across 27 runs. The largest case is a 20-component instance derived from the open-source OpenTitan Earl Grey root-of-trust SoC, targeting a Xilinx Kintex-7 410T FPGA. To establish correctness independently of the ASP solver, an equivalent Constraint Programming (CP-SAT) formulation is built and cross-validated: all 24 synthetic runs (1–8 components) produce identical proven-optimal objectives across both solvers, providing strong evidence of formulation correctness. At 20 components, ASP does not converge to the proven optimum within a 120 s timeout, while CP-SAT proves optimality in under 0.1 s, revealing a qualitative scalability boundary that motivates a hybrid workflow. The benchmark demonstrates constraint sensitivity at realistic scale: latency constraints alone force three high-impact components off zero-trust; tightening LUTs raises risk from 1,804 to 3,449 ($1.91\times$); restricting power forces a mixed security assignment exploiting redundancy-group structure that greedy and local-search heuristics match the optimum under two profiles but miss by 5% under the tightest.

***Index Terms*—design-space exploration, security-aware CAD, System-on-Chip, Answer Set Programming, FPGA, feature synthesis, OpenTitan, cross-solver validation**

I. Introduction

Modern SoCs integrate processors, accelerators, memories, and peripherals on a shared interconnect. A security decision at one component (selecting a zero-trust firewall versus a lightweight MAC checker, for instance) affects system-wide risk exposure, implementation cost, and timing across the device. This makes security architecture a constrained optimization problem: security features, anomaly detectors, and bus monitors improve protection but consume the same LUT, FF, and power budget needed by the functional design.

Prior work has argued that security should become a first-class design objective in embedded-system exploration [1], and several papers have explored design-for-security tradeoffs for specific vulnerabilities or application domains [2]–[8]. What is still missing is (i) a clean, implementation-aware formulation for *system-level* feature synthesis solved to proven optimality, (ii) cross-solver validation establishing that the reported optima are correct, and (iii) evaluation on an architecture-realistic benchmark derived from an open-source SoC at a scale that reveals design insights invisible in small synthetic cases.

We address all three gaps. We formulate security feature synthesis in ASP and solve it with clingo [9]. To validate the results independently, we build an equivalent CP-SAT formulation using Google OR-Tools [10] and cross-validate on every benchmark instance. Our benchmark suite includes eight synthetic cases (1–8 components) and a 20-component instance derived from OpenTitan Earl Grey [11], an open-source silicon root-of-trust SoC with cryptographic accelerators, a key manager, lifecycle controller, and communication peripherals.

Our contributions are fourfold:

1) An exact formulation of system-level SoC security feature synthesis under explicit LUT, FF, power, and per-asset latency constraints, expressed in ASP and solved with clingo.
2) Cross-solver validation: an equivalent CP-SAT model produces identical proven-optimal objectives on all 24 synthetic runs (1–8 components), providing strong evidence of formulation correctness independently of the solver.
3) A scalability comparison from 1 to 20 components showing that ASP proves optimality up to 8 components (2.85 s mean) but fails at 20, while CP-SAT solves every case in under 0.1 s, motivating a hybrid workflow with CP-SAT as the production feature-synthesis solver.
4) A constraint-sensitivity finding on the 20-component OpenTitan case: latency constraints force three components off zero-trust even under full resources; tightening power further forces a mixed 9 ZT + 7 DM + 4 MAC assignment that exploits redundancy-group structure; greedy and local-search heuristics using the same redundancy-aware objective match the optimum under two profiles but miss by 5% under the tightest multi-constraint case.

II. Related Work

Security-aware design-space exploration has been studied for over a decade. In our reading of this literature, most existing work either optimizes for a specific vulnerability or treats security as an auxiliary constraint. Stierand et al. [2]

integrated security requirements into DSE but did not calibrate implementation costs. Gressl et al. [5], [6] explored secure IoT alternatives derived from known attacks. Liu and Li [3], Rajmohan et al. [4], and Linares et al. [7] each study narrower implementation spaces around particular protections. Mahfuz et al. [8] use explainable AI for design-for-security exploration, but target rule-guided hardening rather than exact implementation-aware synthesis.

None of these works (i) solve the system-level feature selection problem to proven optimality under FPGA resource constraints, (ii) provide cross-solver validation of the reported optima, or (iii) evaluate on an architecture-realistic benchmark derived from an open-source SoC.

On the solver side, we note that ASP has been used for hardware configuration [9] and combinatorial optimization. CP-SAT [10] is a state-of-the-art constraint programming solver. Using both on the same problem provides solver-independent correctness evidence that is standard practice in operations research but rare in security-aware CAD.

III. Problem Formulation

A. System Model

We model the input as a set of components $\mathcal{C}$, each containing assets with declared read/write impact scores and per-asset latency caps. Our synthesis engine assigns one security feature $s \in \{\texttt{mac}, \texttt{dynamic_mac}, \texttt{zero_trust}\}$ and one logging feature $\ell \in \{\texttt{no_logging}, \texttt{some_logging}, \texttt{zero_trust_logger}\}$ to each component. Each feature pair (s, ℓ) has known LUT, FF, power, and latency costs drawn from a Vivado-derived catalog, as well as vulnerability (V) and logging (L) scores that determine residual risk.

B. Feature Catalog

We derive feature costs from Vivado 2023.x synthesis of standalone security IP cores (a MAC checker, a dynamic-MAC engine with nonce management, a zero-trust firewall, and three monitoring/logging engines) targeting a Xilinx xc7z020 (PYNQ-Z2 class) at 125 MHz. Zero-trust resource estimates are based on possible implementations explored in Butka and Bobda [12]. LUT and FF counts are architecture-independent 7-series primitive counts and transfer directly to XC7K410T without correction. Power values are Vivado static-power estimates normalised to 10^{-2} W integers; they are order-of-magnitude indicators, not post-place-and-route measurements. The zero-trust logger's large base cost (31,354 LUTs) reflects a full transaction-logging engine with FIFO and formatter.

Each feature has three cost tiers: a *per-asset* cost (proportional to the number of assets on the component), a *per-component* cost (fixed per protected component), and a *base* cost (one-time overhead, shared across all components using that feature). Table I summarizes the catalog.

Lower V and L scores reduce residual risk (Eq. 1), but stronger features cost more resources. Vulnerability scores are scaled by 10 ($V = 10, 20, 30$) to keep all arithmetic in integers. Zero-trust provides the lowest vulnerability ($V = 10$) but requires more LUTs, FFs, and power per component than MAC or dynamic-MAC (Table I), creating a genuine cost–security tradeoff that the optimizer must resolve under each constraint profile. The zero-trust logger, with $L = 5$, cuts the logging factor $4\times$ relative to no-logging ($L = 20$), but at a base cost of 31,354 LUTs. Whether these tradeoffs are worthwhile depends on the system's impact distribution and resource budget; the optimizer answers these questions automatically.

TABLE I
Feature-cost catalog (Vivado synthesis, xc7z020 @ 125 MHz)

Feature	Class	LUTs			FFs			V / L	Pwr[a]
		/asset	/comp	base	/asset	/comp	base		
MAC	sec	51	234	0	32	292	0	$V=30$	2
Dynamic MAC	sec	51	234	1985	32	292	1612	$V=20$	2
Zero Trust	sec	90	380	3200	56	480	2600	$V=10$	3
ZT Logger	log	—	—	31354	—	—	25020	$L=5$	717
Some Logging	log	—	—	3135	—	—	2502	$L=10$	72
No Logging	log	—	—	0	—	—	0	$L=20$	0

Base costs are shared across components that use the same feature.
[a] Power (10^{-2} W): same three-tier model as LUTs (byAsset + byComponent + base).

C. Risk Objective

For each component c, asset a, and operation op, residual risk follows the multiplicative model of Butka and Bobda [12]:

$$r(c, a, op) = \left\lfloor \frac{I(a, op) \cdot V(s_c) \cdot L(\ell_c)}{10} \right\rfloor \tag{1}$$

where I is the declared impact (1–10), V is the vulnerability score of the selected security feature s_c ($V = 10, 20, 30$; lower is better), and L is the logging score ($L = 5, 10, 20$; lower is better). Both V and L are scaled by 10 for integer arithmetic; the divisor compensates, giving effective risk multipliers in $[5, 600]$.

The multiplicative form follows the standard risk = impact $\times$ likelihood decomposition (ISO 27005, NIST SP 800-30): V quantifies residual exposure after the selected prevention mechanism, and L quantifies residual observability gap (a component with full logging has lower undetected-compromise likelihood). The product $V \cdot L$ serves as a proxy for residual likelihood given the chosen feature pair, and multiplication with I yields a composite risk score. We treat V and L as interval-scaled design parameters proportional to residual exposure, giving an *engineering scoring heuristic* suitable for comparative optimization across feature assignments. A per-asset risk cap $r(c, a, op) \leq R_{\max}$ ensures no single asset exceeds an acceptable threshold.

Our objective minimizes total system risk:

$$\min \sum_{c,a,op} r(c, a, op) \tag{2}$$

subject to per-resource constraints (LUTs, FFs, power), per-asset latency caps, and the per-asset risk cap.

D. OpenTitan Earl Grey Benchmark

To move beyond synthetic architectures, we derive a 20-component benchmark from the OpenTitan Earl Grey [11]

TABLE II
OPENTITAN EARL GREY COMPONENT MAPPING (20 IPS)

Comp.	OpenTitan IP	I_r	I_w	Role
cpu	Ibex RISC-V	10	10	Master processor
dma	DMA controller	8	8	Bus master
aes	AES accelerator	9	10	Symmetric crypto
hmac	HMAC	8	9	Authentication
kmac	KMAC	8	9	Keccak MAC
otbn	OTBN bignum	9	10	Asymmetric crypto
keymgr	Key manager	10	10	Root of trust
otp	OTP controller	9	9	Immutable fuses
lc	Lifecycle ctrl	9	8	State transitions
flash	Flash ctrl	7	8	Code/data storage
sram	SRAM ctrl	5	6	Working memory
rom	ROM ctrl	6	3	Boot code
uart0	UART 0	3	3	Serial I/O
uart1	UART 1	3	3	Serial I/O
gpio	GPIO	2	4	General I/O
spi	SPI host	4	5	SPI interface
i2c	I2C	3	4	I2C interface
timer	Timer	2	2	Non-critical
alert	Alert handler	7	5	Security monitor
entropy	Entropy source	8	7	RNG + health

architecture, an open-source silicon root-of-trust SoC designed by lowRISC. Our component set (Table II) maps one-to-one with Earl Grey IP blocks. We assigned impact scores to reflect each block's architectural criticality: cryptographic engines and the key manager receive the highest scores; communication peripherals the lowest. Latency caps reflect each block's timing criticality: the Ibex processor, DMA controller, and key manager require 5-cycle access (forcing MAC), while crypto accelerators allow 7–10 cycles and peripherals are relaxed to 50–1000. The benchmark is "architecture-derived": it preserves OpenTitan's component topology and role differentiation but relies on the external security-IP cost catalog (Table I), not on OpenTitan's actual gate-level implementations.

Two redundancy groups model defense-in-depth coverage: AES/HMAC/KMAC (crypto accelerators that jointly protect key material, so compromise of one does not expose data protected by the others) and UART0/UART1 (redundant communication channels). Group membership asserts overlapping security coverage: combined residual exposure is lower than any single member's.

E. Redundancy Model

For components not in any group, risk follows Eq. 1 directly. For components in a redundancy group $G = \{c_1, \ldots, c_n\}$, the formulation combines per-member residual probabilities before computing risk, modelling independent exposure reduction.

Each member's $V \cdot L$ product is normalised to a common scale using parameters $\mu = 25$ (floor) and $\omega = 1000$ (ceiling):

$$p_{\text{norm}}(c) = \frac{(V_c \cdot L_c - \mu) \cdot 1000}{\omega - \mu} \tag{3}$$

The combined normalised probability is the recursive product:

$$P_G = \frac{p_{\text{norm}}(c_1) \cdot p_{\text{norm}}(c_2) \cdots p_{\text{norm}}(c_n)}{1000^{\,n-1}} \tag{4}$$

which is then denormalised back: $p_{\text{denorm}} = P_G \cdot (\omega - \mu)/1000 + \mu \cdot 10$. Group-member risk becomes $r(c, a, op) = \lfloor I(a, op) \cdot p_{\text{denorm}}/100 \rfloor$. The 1000^{n-1} divisor ensures that adding a redundant member always reduces combined risk, thus giving the optimizer an incentive to accept weaker per-member security when the group already provides coverage.

IV. SOLVER FORMULATIONS

A. ASP Formulation

We encode the problem in ASP and solve it with clingo 5.8.0 [9]. Each component receives exactly one security and one logging feature:

$$\sum_{s \in F_{\text{sec}}} \text{sel_sec}(c, s) = 1, \quad \sum_{\ell \in F_{\text{log}}} \text{sel_log}(c, \ell) = 1 \tag{5}$$

Hard constraints enforce device budgets:

$$\text{LUTs}_{\text{used}} \leq \text{LUTs}_{\max}, \quad \text{Power}_{\text{used}} \leq \text{Power}_{\max} \tag{6}$$

$$\forall (c, a, op): \ \text{latency}(s_c) \leq \text{cap}(a, op) \tag{7}$$

The latency constraint checks only the security feature's cycle cost; logging operates asynchronously and does not add cycles to the protected access path.

Our objective minimizes total residual risk under these constraints. The ASP encoding is declarative: adding a new security feature requires only adding catalog facts, not modifying the solver logic. In our view, this extensibility is the primary reason to choose ASP over imperative formulations.

B. CP-SAT Formulation

To validate the ASP results independently, an equivalent model is built using Google OR-Tools CP-SAT [10]. The CP-SAT model uses:

- **Element constraints** to look up per-pair (security $\times$ logging) resource costs and $V \cdot L$ products from precomputed tables.
- **Multiplication and division equalities** to compute $I \cdot V \cdot L/10$ for each asset.
- **Boolean indicators** with conditional constraints to track one-time base costs for each feature used.

Both formulations encode the identical optimization problem. Any objective mismatch would indicate a bug in one or both encodings.

V. EXPERIMENTAL SETUP

A. Benchmark Suite

Our suite contains nine instance families: eight synthetic cases (TC1–TC9, excluding TC4, which uses a hierarchical component model with a different arity than the flat `component/1` encoding shared by the remaining cases and both solvers) and the OpenTitan-derived case (OT). Table III summarizes the benchmark.

TABLE III
BENCHMARK SUITE

Case	Comp.	Assets	Groups	Target
TC1	3	3	no	Synthetic
TC2	8	8	no	Synthetic
TC3	1	8	no	Synthetic
TC5	8	8	no	Synthetic
TC6	3	3	yes	Synthetic
TC7	8	8	yes	Synthetic
TC8	8	8	yes	Synthetic
TC9	8	8	yes	Synthetic
OT	20	20	yes	OpenTitan Earl Grey

TABLE IV
CONSTRAINT PROFILES

Profile	Target	LUT cap	Power cap	Risk cap
A	xc7z020	53,200	15,000	100
B	xc7z020	10,000	15,000	500
C	xc7z020	53,200	1,000	100
OT-A	XC7K410T	254,200	15,000	200
OT-B	XC7K410T	25,000	15,000	500
OT-C	XC7K410T	254,200	800	200

B. Constraint Profiles

Each synthetic case is evaluated under three profiles (Table IV) targeting a Xilinx xc7z020 (PYNQ-Z2). The OpenTitan case uses three profiles targeting a Xilinx Kintex-7 410T. OT-A provides full resources; OT-B restricts LUTs to 25,000 (below the 45,014 required by the OT-A optimum); OT-C restricts power to 800 (below the 812 required by OT-A), forcing the optimizer to substitute cheaper security features on some components.

C. Experimental Environment

Experiments ran on one P-core of an Intel Core i7-12700H (2.3 GHz) with 32 GB RAM, Windows 11, Python 3.12, clingo 5.8.0, and OR-Tools 9.x. Both solvers were run single-threaded for fair comparison.

VI. RESULTS

A. Cross-Solver Validation

Table V reports the complete benchmark results. All 24 synthetic runs (1–8 components) produce identical proven-optimal objectives between ASP and CP-SAT, confirming that both formulations encode the same optimization problem. At 20 components (OpenTitan), CP-SAT proves optimality in under 0.1 s, but ASP does not converge to the proven optimum within its 120 s timeout, confirming a qualitative scalability boundary between 8 and 20 components.

B. Scalability Analysis

Table VI aggregates solve time by component count. At 1 component, both solvers finish in under 2 ms. At 3 components, ASP takes 0.12 s (dominated by grounding), while CP-SAT stays under 12 ms. At 8 components, ASP averages 2.85 s while CP-SAT remains under 25 ms, a 220× speedup.

TABLE V
BENCHMARK RESULTS: 24 CROSS-VALIDATED SYNTHETIC + 3 OPENTITAN

Case	Comp.	Prof.	Obj.	ASP (s)	CP-SAT (s)	Match
TC1	3	A	90	0.125	0.010	✓
TC1	3	B	180	0.116	0.006	✓
TC1	3	C	90	0.117	0.005	✓
TC2	8	A	365	2.424	0.007	✓
TC2	8	B	860	3.616	0.011	✓
TC2	8	C	365	2.579	0.007	✓
TC3	1	A	720	0.002	0.001	✓
TC3	1	B	1,440	0.001	0.001	✓
TC3	1	C	720	0.001	0.001	✓
TC5	8	A	365	2.424	0.006	✓
TC5	8	B	860	3.436	0.011	✓
TC5	8	C	365	2.559	0.007	✓
TC6	3	A	45	0.107	0.015	✓
TC6	3	B	45	0.101	0.012	✓
TC6	3	C	45	0.134	0.018	✓
TC7	8	A	232	2.534	0.021	✓
TC7	8	B	402	3.226	0.023	✓
TC7	8	C	232	2.530	0.015	✓
TC8	8	A	331	2.538	0.010	✓
TC8	8	B	761	3.337	0.017	✓
TC8	8	C	331	2.537	0.010	✓
TC9	8	A	219	2.821	0.016	✓
TC9	8	B	364	3.337	0.024	✓
TC9	8	C	219	3.098	0.018	✓
OT	20	OT-A	1,804	>165†	0.05	—
OT	20	OT-B	3,449	>296†	0.05	—
OT	20	OT-C	1,854	>213†	0.05	—

† ASP suboptimal; objectives are CP-SAT proven optima.

TABLE VI
SCALABILITY: MEAN SOLVE TIME BY COMPONENT COUNT

Components	ASP mean (s)	CP-SAT mean (s)	Ratio
1	0.001	0.001	1×
3	0.12	0.012	10×
8	2.85	0.013	220×
20	>225†	0.05	>4,500×

† ASP did not prove optimality; best-found was suboptimal.

At 20 components (OpenTitan), ASP does not converge to the proven optimum within its timeout, while CP-SAT proves optimality in under 0.1 s. The scalability boundary between 8 and 20 components marks where CP-SAT becomes essential for practical use.

Figure 1 plots the scaling trend on a log scale. The gap between solvers widens steadily: 1× at 1 component, 10× at 3, 220× at 8, and >4,500× at 20 where ASP fails to prove optimality entirely. For rapid parameter sweeps or larger designs, we recommend CP-SAT as the feature-synthesis solver; ASP's value lies in the declarative extensibility needed by

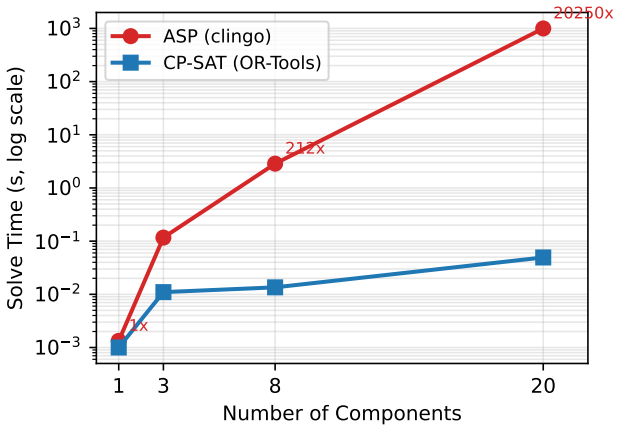

Fig. 1. Mean solve time vs. component count (log scale). The gap widens from 1× at 1 component to >4,500× at 20 components, where ASP cannot prove optimality.

TABLE VII
OPENTITAN EARL GREY RESULTS (20 COMPONENTS, CP-SAT PROVEN OPTIMA)

Profile	Obj.	LUTs	Power	Security mix	Logging mix
OT-A	1,804	45,014	812	15 ZT+2 DM+3 MAC	20 ZTL
OT-B	3,449	16,610	165	14 ZT+3 DM+3 MAC	20 SL
OT-C	1,854	43,904	800	9 ZT+7 DM+4 MAC	20 ZTL

ZT=Zero Trust, DM=Dynamic MAC, MAC=MAC, ZTL=ZT Logger, SL=Some Logging

downstream reasoning stages (Section VII-A).

C. OpenTitan Design Insights

Table VII reports the OpenTitan results in detail.

Under Profile OT-A (full resources), latency constraints already force the optimizer to differentiate: cpu, dma, and keymgr have 5-cycle latency caps that exclude zero-trust (8 cycles), so they receive MAC; aes and otbn have 7-cycle caps that exclude ZT but allow dynamic-MAC. The remaining 15 components receive zero-trust. Risk is 1,804 at 45,014 LUTs and 812 power. Even with non-binding resource budgets, latency alone prevents a uniform ZT assignment.

Under OT-B (LUT cap 25,000), the ZTL base cost (31,354 LUTs) exceeds the budget, so the optimizer switches all logging to some-logging. Latency constraints still force cpu/dma/keymgr to MAC, and hmac joins the dynamic-MAC group. Risk rises to 3,449.

The most interesting result is OT-C (power capped at 800). Here the optimizer must navigate power constraints, latency constraints, and redundancy structure simultaneously. It maintains full ZTL logging but downgrades security on 11 of 20 components: 4 receive MAC and 7 receive dynamic-MAC, raising risk to 1,854. Surprisingly, the three crypto accelerators (aes, hmac, kmac) all receive dynamic-MAC or MAC despite their high individual impact. Their redundancy group membership reduces combined probability enough that ZT's risk saving is less valuable than the power saved by a weaker feature. An impact-ordered greedy would assign ZT to crypto engines first, yet the optimizer exploits redundancy coverage instead. This kind of multi-constraint, redundancy-aware assignment benefits from exact optimization.

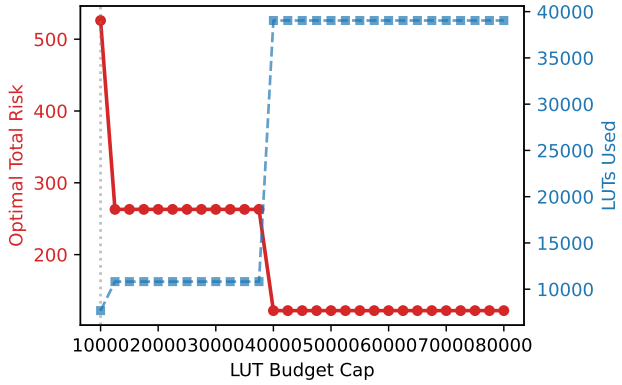

Fig. 2. Pareto sweep: optimal risk vs. LUT budget cap on the OpenTitan case. Sharp transitions correspond to security-tier thresholds (all-MAC → DM mix → ZT mix); mixed security assignments appear under tight budgets.

TABLE VIII
HEURISTIC VS. OPTIMAL (CP-SAT) ON OPENTITAN PROFILES

Profile	Optimal	Greedy	Gap_G	Local-S	Gap_{LS}
OT-A	1,804	1,804	0%	1,804	0%
OT-B	3,449	3,449	0%	3,449	0%
OT-C	1,854	1,948	+5%	1,948	+5%

D. Heuristic Baseline Comparison

To quantify what exactness buys, Table VIII compares CP-SAT optima against two heuristic baselines: a *greedy* heuristic that processes components by descending impact and assigns the best affordable feature, and a *local-search* (1-opt) refinement that starts from the greedy solution and iteratively swaps one component's security/logging assignment to the best feasible improving neighbour until no single-component swap reduces risk.

Both heuristics use the same redundancy-aware objective as CP-SAT (Section III-E), so the comparison is apples-to-apples. The greedy assigns the lowest-risk latency-feasible pair to each component in descending impact order, checking resource caps incrementally. Local search (1-opt) then swaps one component at a time to the best feasible neighbour.

Under OT-A and OT-B, both heuristics match the proven optimum: the greedy's impact-ordered processing happens to find the same assignment the solver proves optimal. Under OT-C, the gap is 5%: the simultaneous power and latency pressure creates a multi-component coordination problem where the greedy's sequential decisions cannot find the globally optimal trade between crypto-group coverage and power savings.

E. Perturbation Stability

All 24 synthetic runs produce identical proven optima across ASP and CP-SAT, validating the formulation on 1–

8 component instances. Profile C always reproduces Profile A at 8 components (power is never binding), whereas on OpenTitan, OT-C produces a dramatically different solution with mixed security assignments. Power sensitivity and cost differentiation emerge only at realistic scale.

VII. Discussion

A. Why ASP for Security Feature Synthesis

At 8 components CP-SAT is 220× faster than ASP; at 20 components ASP cannot prove optimality at all, while CP-SAT completes in under 0.1 s (Table VI). For feature-synthesis alone, CP-SAT is the production solver. ASP's value rests on the downstream reasoning stages. Feature synthesis is the first of three stages; the maintained repository also implements *access-control policy synthesis* (firewall placement, least-privilege gap analysis via negation-as-failure) and *resilience analysis* (parallel transitive closures over degraded topologies, bounded-depth attack-path enumeration). Both stages rely on recursive reachability, negation-as-failure, and multi-level implication chains, patterns natural in ASP but costly in CP-SAT or ILP. A hybrid workflow makes sense: CP-SAT for feature-synthesis optimization, ASP for the downstream stages.

B. Relation to ILP/MIP Solvers

Since the resource constraints are linear and the objective can be linearised, so ILP solvers (e.g. Gurobi [13]) could solve the same problem. We chose CP-SAT because it is open-source and its element constraints map naturally to feature selection; an ILP comparison remains future work.

C. Constraint Sensitivity as a Design Tool

We see the OT results as a practical demonstration of exact DSE's value: quantifying the security cost of constraint interactions. Even under full resources (OT-A), latency alone raises risk from a hypothetical all-ZT floor to 1,804 by forcing three components to MAC. Restricting LUTs to 25,000 (OT-B) eliminates ZTL logging and tightens the security mix further, pushing risk to 3,449. Power at 800 units (OT-C) forces a different trade: the optimizer keeps ZTL but downgrades security on 11 components to a 9 ZT + 7 DM + 4 MAC mix at risk 1,854. The Pareto sweep (Figure 2) provides a complete risk–resource tradeoff map. Quantitative what-if analysis at this level requires an exact solver that reports provably optimal solutions under each constraint set.

D. Limitations

We assigned impact scores based on architectural criticality, not measured vulnerability data. Our feature catalog comes from Vivado synthesis of standalone security IP cores (MAC checker, dynamic-MAC engine, zero-trust firewall, and three logging monitors), not the actual OpenTitan blocks. The formulation covers the feature-synthesis stage; the policy and resilience stages are implemented in the maintained repository. Scaling beyond 20 components to 50–100 component SoCs remains future work.

VIII. Conclusion

We set out to answer a straightforward question: can security feature synthesis for SoCs be solved exactly, and does exactness actually matter compared to reasonable heuristics? The answer to both is yes, but the path taught us more than the destination.

Cross-validating ASP against CP-SAT on 24 synthetic runs gave us confidence that the formulation is correct, not just that one solver happens to find good solutions. The perfect agreement across 24 synthetic runs was reassuring; the sharp divergence at 20 components was informative. We now view CP-SAT as the right production solver for feature synthesis, while ASP earns its keep in the downstream policy and resilience stages where recursive reasoning dominates.

The result we find most valuable is how latency and power interact. Under OT-A, latency alone forces cpu, dma, and keymgr off zero-trust, producing a heterogeneous assignment even with non-binding resource budgets. Under OT-C, adding a power constraint forces the optimizer to downgrade 11 of 20 components, producing a 9 ZT + 7 DM + 4 MAC mix where the crypto accelerators receive weaker features because redundancy coverage absorbs the risk. Conventional priority ordering would protect crypto engines first; the optimizer correctly exploits group structure instead.

Looking ahead, we plan to co-optimize firewall and policy placement with feature selection, evaluate resilience under component-compromise scenarios on the OpenTitan topology, and add an ILP solver as a third comparison point.

Our ASP solver, CP-SAT benchmark, feature catalog, and all benchmark instances are available in the accompanying repository.[1]

References

[1] A. D. Pimentel, "A case for security-aware design-space exploration of embedded systems," *Journal of Low Power Electronics and Applications*, vol. 10, no. 3, p. 22, 2020.

[2] I. Stierand, S. Malipatlolla, S. Froeschle, A. Stuehring, and S. Henkler, "Integrating the security aspect into design space exploration of embedded systems," in *Proceedings of the IEEE International Symposium on Software Reliability Engineering Workshops*, 2014, pp. 371–376.

[3] Q. Liu and H. Li, "Hardware design space exploration with a new dimension - ip protection robustness," in *Proceedings of the Euromicro Conference on Digital System Design*, 2015, pp. 599–605.

[4] S. Rajmohan, N. Ramasubramanian, and N. Naganathan, "Hybrid evolutionary design space exploration algorithm with defence against third party ip vulnerabilities," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2602–2614, 2020.

[5] L. Gressl, C. Steger, and U. Neffe, "Security driven design space exploration for embedded systems," in *Proceedings of the Forum on Specification and Design Languages*, 2019, pp. 1–8.

[6] ——, "Design space exploration for secure iot devices and cyber-physical systems," *ACM Transactions on Embedded Computing Systems*, vol. 20, no. 4, pp. 1–24, 2021.

[7] A. Linares, D. Hely, F. Lhermet, and G. D. Natale, "Design space exploration applied to security," in *Proceedings of the International Conference on Design and Technology of Integrated Systems in Nanoscale Era*, 2021, pp. 1–4.

[8] T. Mahfuz, S. Bhunia, and P. Chakraborty, "X-dfs: Explainable artificial intelligence guided design-for-security solution space exploration," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 753–766, 2025.

[1] Repository URL withheld for double-blind review.

[9] M. Gebser, B. Kaufmann, R. Kaminski, M. Ostrowski, T. Schaub, and M. Schneider, "Potassco: The potsdam answer set solving collection," *AI Communications*, vol. 24, no. 2, pp. 107–124, 2011.

[10] Google OR-Tools, "OR-Tools: Google's operations research tools," https://developers.google.com/optimization, 2026, cP-SAT constraint programming solver; accessed Apr. 3, 2026.

[11] lowRISC contributors, "OpenTitan: Open source silicon root of trust," https://opentitan.org, 2026, earl Grey top-level SoC with Ibex core, AES, HMAC, KMAC, OTBN, key manager, OTP, lifecycle controller, and TL-UL crossbar; accessed Apr. 3, 2026.

[12] B. Butka and C. Bobda, "Measuring system-on-chip bus security using a zero-trust risk assessment," *IEEE Embedded Systems Letters*, 2025, early access, doi: 10.1109/LES.2025.XXXXXXX.

[13] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," https://www.gurobi.com, 2026, accessed Apr. 4, 2026.